Security of ZisK[1]

Héctor Masip-Ardevol
hector@zisk.technology
ZisK Core Team

September 14, 2026

Abstract

This document analyzes the security of ZisK's proof system, from the proof of a single AIR to the statement about program execution that a final proof certifies. It is organized in three parts that compose. Part I describes the proving system as implemented and bounds the knowledge soundness of every argument it uses. Part II analyzes the recursive aggregation layer: the bus argument that ties the AIR instances together is abstracted as an additive fingerprint of a weighted multiset of messages, and the per-node bookkeeping as accumulators over a commutative monoid, and any adversary producing an accepting Final proof is turned into an extractor that outputs a bounded collection of locally satisfying, bus-balanced AIR witnesses. Part III shows that such a collection encodes a successful execution of the committed program. The composition yields the knowledge soundness of the final verifier for ZisK's execution relation, with an explicit error bound. The idealizations made, the assumptions inherited from the primitives and the obligations left open are recorded in a ledger.

[1]This document describes ZisK v0.16.1 [48cf7cc – 2026-20-03].
Source code available at https://github.com/0xPolygonHermez/zisk.

Contents

1. Introduction **2**
1.1. What ZisK Proves . . . 2
2. Preliminaries **4**
2.1. Notation and Parameters . . . 4
2.2. Arithmetization . . . 4
2.3. Interfaces . . . 8
2.4. Security Definitions . . . 8
3. The ZisK Proof System **10**
3.1. Structure . . . 10
3.2. Accumulators . . . 12
3.3. Recursive Relations . . . 13
3.4. ZisK Instantiation . . . 15
References **17**

1. Introduction

Consider a party that needs the result of a computation but cannot, or does not want to, carry it out itself: a blockchain that must validate a block of transactions without every node re-executing it, a light client that follows a chain it cannot store, or a service that runs a computation on behalf of its users. The party delegates the computation to a *prover*, which runs the program and returns the result. Nothing guarantees that the prover is honest, so the result comes with a proof, and the party that checks it, the *verifier*, accepts the result only if the proof convinces it of the claim

"The program P, run on the input x, produces the output y."

A *zero-knowledge virtual machine* (zkVM) produces such proofs for arbitrary programs, written in ordinary programming languages and compiled to a standard instruction set, rather than for computations written by hand as constraints. ZisK [Zis26] is such a system, for programs compiled to RISC-V and translated into the ZisK instruction set. Its prover does not produce one monolithic proof of the entire execution. ZisK splits the work of the VM among a fixed family of *chips*, each in charge of one kind of operation (e.g., the main loop, memory or the arithmetic), and splits the execution of each chip into segments; every segment is proved by a succinct argument for its chip; and the proofs are aggregated by a tree of recursive verifiers whose root checks that the segments fit together. The verifier receives a single proof, the *final proof*, together with the public statement it attests to.

The security of such a system is not the security of its textbook components. Each layer departs from the construction whose analysis it would like to inherit: the interactive oracle proof (IOP) behind every argument is made non-interactive with a transcript that does not observe everything the verifier reads, the segment proofs share a bus key derived once for the whole execution, the recursive verifiers switch verification on and off depending on the shape of the tree, and the bus contributions of the segments are checked only once, summed, at the root. A gap in any of these places can make an accepting proof certify nothing, while every component remains sound on its own. This paper states precisely what an accepting final proof of ZisK certifies, and proves it for the protocol as deployed, under explicit assumptions on the primitives.

1.1. What ZisK Proves.

The claim above cannot be what the final proof certifies as it stands. Delegation pays off only if checking the proof is much cheaper than running the program, and to check the claim the verifier would have to read P and x. Moreover, x is often the part of the computation that the prover wants to keep private. ZisK therefore proves a refinement of the claim, obtained by revisiting each of its three components and adding a fourth.

The program. ZisK proves executions of programs for the ZisK instruction set, a machine with registers and a byte-addressed memory, into which RISC-V programs are translated; a program starts from a fixed initial state. A program can be megabytes long, so the verifier does not receive P but a short *commitment* to it, a value computed from its code such that no one can find a different program with the same value. A verifier that knows which program it expects, for instance because its developer published the commitment, compares the commitment with that value, and the proof then speaks about that program and no other.

The input. The input x is supplied before the program starts. It may be large, such as a batch of transactions, or secret, such as a signing key, and in either case it should not be sent to the verifier. The claim therefore does not fix x but quantifies over it: the proof says that *some* input

leads to the output. Whatever the verifier must know about x has to be revealed by the program itself, for instance by outputting a hash of it.

The output. A program has no single output: when it halts, all of its memory is a candidate. ZisK lets the program *declare public outputs* while it runs, the values it chooses to reveal, and y is the sequence of these values.

The hints. To run efficiently, a program may request values from the prover while it runs, its *hints*, typically the result of a costly computation that the program then checks, such as a square root that it verifies by squaring. Hints do not appear in the original claim and come from an untrusted source, so the claim quantifies over them as it does over the input. Neither kind of data is trusted: the program must behave correctly whatever values it receives, and checking the hints is the responsibility of the program, not of ZisK.

The public statement. A proof is therefore checked against a short *public statement* $\boldsymbol{p}$ made of two parts: the public outputs y and the commitment to P. The input and the hints stay with the prover. What a proof claims is:

> *There exist an input x and a sequence of hints on which the committed program P, started from its initial state, runs according to the semantics of the ZisK instruction set until it halts, and declares exactly the public outputs y of $\boldsymbol{p}$.*

We formalize this claim as a relation $\mathcal{R}_{\text{exec}}$ whose witness is the whole execution: the program, the input, the hints and the sequence of machine states (Part III). The claim concerns the committed ZisK program; its connection to the original RISC-V binary rests on the correctness of the transpiler, which we assume.

Theorem 1.1 (Main Theorem, Informal). *For every efficient adversary that makes at most Q queries to the random oracles and outputs $\boldsymbol{p}$ with a final proof that the verifier accepts, an extractor outputs a witness of $\mathcal{R}_{\text{exec}}$ for $\boldsymbol{p}$, except with probability at most the sum of:*

- *The knowledge-soundness errors of the arguments proving the nodes of the aggregation tree, each at most Q times the largest round-by-round error of its IOP.*
- *The soundness error of the bus argument at a key derived after the leaves are committed, for each of the Q queries.*
- *The extraction errors of the challenge accumulator and of the commitments.*

The proof of Theorem 1.1 retraces, in reverse, the way ZisK builds a proof: from the final proof it recovers a proof for every segment, from each of these proofs a valid trace of its segment, and from the traces an execution of the program (Figure 1).

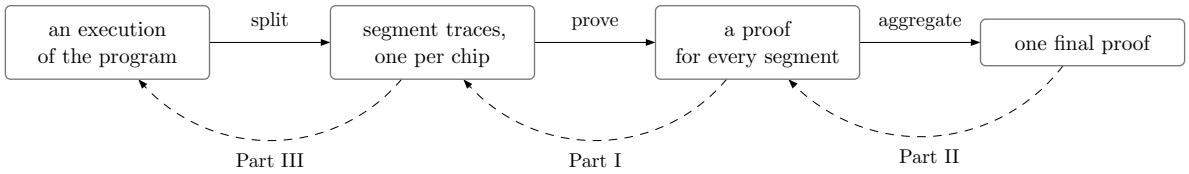


FIGURE 1. How ZisK proves an execution (solid arrows) and how the analysis recovers one from an accepted final proof (dashed arrows). Part IV composes the three steps.

Part I: The Proving System. Every chip and every recursive verifier of ZisK is proved with the same protocol: an IOP that checks the constraints of its chip, compiled into a succinct argument with Merkle trees and the Fiat–Shamir transformation. The compilation is where ZisK departs

from the textbook: the bus key and the first stage of every chip proof are bound through the global challenge rather than through the proof's own transcript. A prover that could choose its first-stage columns after the out-of-domain sample could satisfy the constraint check with any trace, so the binding lemma, which combines preimage awareness of the accumulator with extractability of the Merkle trees, is what makes the chip proofs sound.

Part II: The Recursion. Each node of the aggregation tree carries a leaf counter, a homomorphic hash of the leaves' typed commitments, and the sum of their bus contributions; the root of the tree checks that the bus key was derived from the hash and that the contributions, with the global messages of the statement, sum to zero. The analysis descends the tree with one extraction game per layer, and a zero test at a key derived after the leaves are fixed turns the root check into balance of every bus.

Part III: The Semantics. A balanced collection of locally satisfying witnesses of the ZisK chips encodes an execution: the memory argument reconstructs the memory, the continuation buses stitch the segments of each chip into one sequence, and the constraints of the Main chip and of the chips it delegates to implement one step of the instruction set. The decoding is deterministic and involves no cryptography.

Part IV: Composition. The last part composes the three steps into the main theorem and records, in a ledger, every assumption the theorem rests on and every obligation left open.

2. Preliminaries

This section fixes the notation and the definitions shared by the three parts of the paper: the arithmetization layer (AIRs, interactions and buses), the interfaces of the cryptographic components (trace commitments, the multiset hash and key derivation) and the security notions they are required to satisfy. ZisK's concrete choices are collected in Section 3.4.

2.1. Notation and Parameters.

Let $\mathbb{F}$ be a prime field of order p and $\mathbb{K}$ an extension field of $\mathbb{F}$. $\mathbb{Z}_p^n$ denotes the additive group of n -vectors of integers modulo p, the same set as $\mathbb{F}^n$ viewed as a $\mathbb{Z}$-module. For integers $a \le b$, $[a, b]$ and $[a, b)$ are the closed and half-open integer intervals and $[b] := [1, b]$. Field elements are identified with integers in $[0, p)$ when needed. λ is the security parameter; "PPT" means probabilistic polynomial time in λ, and $\mathsf{negl}(\lambda)$ and $\mathsf{poly}(\lambda)$ denote negligible and polynomially bounded functions. All algorithms have oracle access to the random oracles introduced in Section 2.3; Q bounds the total number of oracle queries an adversary makes.

The recursion is governed by two integer parameters: the *aggregation arity* $A \ge 2$, the number of children an aggregation node can take, and the *leaf-count exponent* $B \ge 1$: an aggregation tree may contain at most $2^B - 1$ leaves. We assume throughout that

$$A \cdot 2^B \le p, \quad 2^B < p, \quad 2^B \le \mathsf{poly}(\lambda) :$$

the first condition guarantees that sums of A range-checked counters never wrap around in $\mathbb{F}$, the second is the cardinality restriction required by the multiset hash, and the third makes the number of leaves, and hence the running time of the extractor, polynomial.

2.2. Arithmetization.

A ZisK proof covers many *instances* of a fixed family of chips. The constraints of an instance read two kinds of values: the *columns* of its trace, which take one value per row, and the *scalar values*, which are constant over the rows.

Definition 2.1 (AIR and Witness). Given integers $N \geq 1$, $W \geq 1$ and $V \geq 0$ and a finite set $O \subset \mathbb{Z}$ of *row offsets*, an *algebraic intermediate representation* (AIR) of *height* N is a finite set of *constraints*

$$\mathcal{C} \subset \mathbb{K}\Big[X_j^{(o)}, Y_l : j \in [W], o \in O, l \in [V]\Big].$$

The variables $X_j^{(o)}$ refer to the columns and the variables Y_l to the scalar values. A *witness* for $\mathcal{C}$ is a pair $w = (\boldsymbol{T}, \boldsymbol{v})$ with $\boldsymbol{T} \in \mathbb{K}^{N \times W}$, the *trace*, and $\boldsymbol{v} \in \mathbb{K}^V$, the *values*. At row $r \in [N]$ the variable $X_j^{(o)}$ takes the value $T_j(r+o)$, with row indices taken modulo N, and the variable Y_l the value v_l. The witness *satisfies* $\mathcal{C}$ if every constraint polynomial evaluates to zero at every row.

Offsets may be negative: $X_j^{(o)}$ with $o < 0$ reads row $r - |o|$. Since the rows are taken modulo N, the constraints are enforced on a cyclic domain, and shifting every constraint by $\min O$ rows gives an equivalent AIR with offsets in $[0, \max O - \min O]$; we keep the offsets as the AIR is written.

The columns and the values of a witness are of several kinds, which differ in who chooses them, who sees them and how many instances share them. The description of an AIR partitions both.

Definition 2.2 (Column Classes). The description of an AIR $\mathcal{C}$ with W columns and height N partitions its column indices as $[W] = J^{\mathrm{fix}} \sqcup J^{\mathrm{cach}} \sqcup J^{\mathrm{wit}}$:

1. The *fixed columns* J^{fix}, whose values are prescribed as part of the description of $\mathcal{C}$ and hence known to both the prover and the verifier.
2. The *cached columns* J^{cach}, whose values are computed once and reused across different proofs. Their values are not part of the description; instead, a commitment to them (Definition 2.11) travels in the publics, at the coordinates designated by a *cached-commitment binding* $\gamma : [m] \to [k]$.
3. The *witness columns* J^{wit}, computed afresh for every proof and only known in full to the prover.

The *prescribed data* of $\mathcal{C}$ is the family $\boldsymbol{d} = \big(d_j\big)_{j \in J^{\mathrm{fix}}}$ of fixed columns $d_j \in \mathbb{F}^N$. For a trace $\boldsymbol{T}$ and a set of columns J we write $\boldsymbol{T}|_J$ for the submatrix of the columns in J.

Definition 2.3 (Value Classes). The description of an AIR $\mathcal{C}$ with V values partitions its value indices as $[V] = I^{\mathrm{pub}} \sqcup I^{\mathrm{pv}} \sqcup I^{\mathrm{air}} \sqcup I^{\mathrm{grp}}$, with $\big|I^{\mathrm{pub}}\big| = k$ and $|I^{\mathrm{pv}}| = k'$ for every AIR of a proof. For values $\boldsymbol{v}$ and a set of indices I we write $\boldsymbol{v}|_I$ for the subvector of the coordinates in I.

1. The *publics* $\boldsymbol{p}_{\mathrm{ext}} := \boldsymbol{v}|_{I^{\mathrm{pub}}} \in \mathcal{P}_{\mathrm{ext}} := \mathbb{F}^k$, shared by all instances and seen by the external verifier.
2. The *proof values* $\boldsymbol{p}_{\mathrm{pv}} := \boldsymbol{v}|_{I^{\mathrm{pv}}} \in \mathcal{P}_{\mathrm{pv}} := \mathbb{F}^{k'}$, chosen by the prover once per proof and shared by all instances, but not seen by the external verifier.
3. The *AIR values* $\boldsymbol{a} := \boldsymbol{v}|_{I^{\mathrm{air}}}$ of an instance, chosen by the prover for that instance.
4. The *airgroup values* $\boldsymbol{s} := \boldsymbol{v}|_{I^{\mathrm{grp}}}$ of an instance, that its proof exposes and that are aggregated, with a fixed operation, over all the instances of a group of chips called an *airgroup*.

The publics and the proof values form the *public statement*

$$\boldsymbol{p} = \big(\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}}\big) \in \mathcal{P} := \mathcal{P}_{\mathrm{ext}} \times \mathcal{P}_{\mathrm{pv}}.$$

Every AIR that takes part in a proof reads the whole public statement, but the external verifier sees only $\boldsymbol{p}_{\mathrm{ext}}$: the proof values are hidden by the root of the aggregation tree (Section 3). A *global constraint* is a identity over the aggregated airgroup values and the public statement, , checked once per proof rather than by any instance.

The AIRs of the ZisK chips, as modelled in this paper, read only the AIR values, the publics and the proof values; their constraints are therefore satisfied or not under a public statement $\boldsymbol{p}$. Airgroup values are read only by the constraints that the proving system adds to every AIR to realize its interactions, whose coefficients include the challenges of the bus argument (Part I).

Definition 2.4 (Interaction). An *interaction* of an AIR $\mathcal{C}$ on the bus $b \in \mathbb{F}$ is a tuple $\iota = (b, m_\iota, \boldsymbol{e}_\iota)$, where the *multiplicity* m_ι and the *message* components $e_{\iota,1}, ..., e_{\iota,\ell_b}$ are polynomials over $\mathbb{F}$ in the variables of $\mathcal{C}$. Given a witness w and a row r, we write $m_\iota(w, r) \in \mathbb{F}$ and $\boldsymbol{e}_\iota(w, r) \in \mathbb{F}^{\ell_b}$ for their evaluations at row r. The set of bus identifiers in use is $\mathcal{B}$; every interaction on bus b has the common message length ℓ_b, and $\ell := \max_{b \in \mathcal{B}} \ell_b$.

Definition 2.5 (Chip). A *chip* is a pair $(\mathcal{C}, \mathcal{I})$ of an AIR $\mathcal{C}$ and a finite set $\mathcal{I}$ of interactions of $\mathcal{C}$.

A chip is the unit into which ZisK splits the work of the VM: the main loop, each memory region, each arithmetic unit and each precompile is a chip. Throughout, we fix a positive integer M and an indexed family of chips

$$\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau \in [M]},$$

where τ is the *type* of the chip and $\mathcal{C}_\tau$ has height N_τ and parameters W_τ, O_τ and V_τ. We write $\mathcal{W}_\tau := \mathbb{K}^{N_\tau \times W_\tau} \times \mathbb{K}^{V_\tau}$ for the witness space of type τ.

Each chip carries its column and value classes and its prescribed data; we index them by the type, as in J_τ^{fix}, I_τ^{air}, $\boldsymbol{d}_\tau$ and γ_τ.

Definition 2.6 (Local Satisfaction). For $\tau \in [M]$, $\boldsymbol{p} = (\boldsymbol{p}_{\text{ext}}, \boldsymbol{p}_{\text{pv}}) \in \mathcal{P}$ and $w = (\boldsymbol{T}, \boldsymbol{v}) \in \mathcal{W}_\tau$, the predicate $\text{Local}_\tau(\boldsymbol{p}, w)$ holds if and only if:

1. w satisfies $\mathcal{C}_\tau$.
2. The values carry the public statement: $\boldsymbol{v}|_{I_\tau^{\text{pub}}} = \boldsymbol{p}_{\text{ext}}$ and $\boldsymbol{v}|_{I_\tau^{\text{pv}}} = \boldsymbol{p}_{\text{pv}}$.
3. $\boldsymbol{T}$ agrees with the prescribed data on the fixed columns: $T_j(r) = d_{\tau,j}(r)$ for every $j \in J_\tau^{\text{fix}}$ and $r \in [N_\tau]$.
4. The cached columns open the commitment carried in $\boldsymbol{p}_{\text{ext}}$:

$$\mathsf{Commit}_{\text{cach}}\big(\boldsymbol{T}|_{J_\tau^{\text{cach}}}\big) = \big(p_{\gamma_\tau(1)}, ..., p_{\gamma_\tau(m)}\big),$$

   where p_i is the i-th coordinate of $\boldsymbol{p}_{\text{ext}}$.

Fixed columns are part of the description of $\mathcal{C}_\tau$. Cached columns are not: they are computed once per program, committed with $\mathsf{Commit}_{\text{cach}}$, and the commitment travels in the publics, where the description of $\mathcal{C}_\tau$ designates its coordinates. The program being executed is therefore bound through $\boldsymbol{p}$ rather than hard-wired into the AIRs: the program ROM and the boot state live in cached columns of the relevant chips, and whoever accepts a Final proof for $\boldsymbol{p}_{\text{ext}}$ must check that the cached commitments carried in $\boldsymbol{p}_{\text{ext}}$ are those of the intended program.

Definition 2.7 (Message Multiset of a Witness). Let $\mathcal{V} := \big\{(b, v) : b \in \mathcal{B}, v \in \mathbb{F}^{\ell_b}\big\}$ be the set of *bus messages*. An $\mathbb{F}$-*weighted multiset* of messages is a finitely supported map $D : \mathcal{V} \to \mathbb{F}$; we write $|D|$ for the size of its support and add such maps pointwise. For $\tau \in [M]$ and $w \in \mathcal{W}_\tau$, the *message multiset* of w is

$$\text{Msg}_\tau(w)(b, v) := \sum_{\iota \in \mathcal{I}_\tau, b_\iota = b} \sum_{r=1}^{N_\tau} m_\iota(w, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w, r) = v].$$

Its support has at most $N_\tau \cdot |\mathcal{I}_\tau|$ elements.

A row that sends a message with positive multiplicity and a row of the same witness that receives it with negative multiplicity cancel in $\mathrm{Msg}_\tau(w)$: the multiset records the net flow of every message out of the witness.

Definition 2.8 (Bus Fingerprint). A *bus fingerprint* $\mathcal{F}$ consists of a finite *key space* $\mathcal{X}$, an abelian group $G_{\mathcal{F}}$ of *values*, a class $\mathcal{D}$ of *admissible* nonzero $\mathbb{F}$-weighted multisets of messages, and, for every key $\chi \in \mathcal{X}$, a map $\mathcal{F}_\chi$ from $\mathbb{F}$-weighted multisets of messages to $G_{\mathcal{F}} \cup \{\perp\}$, such that:

1. (*Additivity*) $\mathcal{F}_\chi(0) = 0$, and $\mathcal{F}_\chi(D + D') = \mathcal{F}_\chi(D) + \mathcal{F}_\chi(D')$ whenever the right-hand side is defined;
2. (*Soundness*) For every Λ and every adversary $\mathcal{A}$ that outputs a multiset before the key is drawn uniformly from $\mathcal{X}$, the *fingerprint advantage*:

$$\mathsf{Adv}^{\mathsf{fp}}_{(\mathcal{F},\Lambda)}(\mathcal{A}) := \Pr\big[D \leftarrow \mathcal{A}; \chi \leftarrow \mathcal{X} : D \in \mathcal{D} \wedge |D| \le \Lambda \wedge \mathcal{F}_\chi(D) = 0\big]$$

is small. The fingerprint is *statistically sound* with error $\varepsilon_{\mathcal{F}}$ if $\mathsf{Adv}^{\mathsf{fp}}_{(\mathcal{F},\Lambda)}(\mathcal{A}) \le \varepsilon_{\mathcal{F}}(\Lambda)$ for every adversary, with $\varepsilon_{\mathcal{F}}$ non-decreasing, and *computationally sound* if the advantage is negligible for every PPT adversary.

The *bus contribution* of $w \in \mathcal{W}_\tau$ under the key χ is

$$\mathrm{BusCont}_\chi(\mathcal{I}_\tau, w) := \mathcal{F}_\chi(\mathrm{Msg}_\tau(w)) \in G_{\mathcal{F}} \cup \{\perp\}.$$

A fingerprint is an additive summary of a multiset that vanishes on an admissible nonzero multiset only for few keys, or only for keys no efficient adversary can anticipate.

Definition 2.9 (Typed Witness Collection). A *typed witness collection* is a finite list

$$L = \big((\tau_1, w_1), ..., \big(\tau_{|L|}, w_{|L|}\big)\big), \quad \tau_i \in [M], \quad w_i \in \mathcal{W}_{\tau_i}.$$

Types and witnesses may repeat, and each entry counts separately; the order of the list carries no meaning. The *number of bus terms* of L is $\Lambda(L) := \sum_{i=1}^{|L|} N_{\tau_i} \cdot \big|\mathcal{I}_{\tau_i}\big|$, and we set $\Lambda_{\max} := (2^B - 1) \cdot \max_{\tau \in [M]} N_\tau |\mathcal{I}_\tau| + g$, so that $\Lambda(L) + g \le \Lambda_{\max}$ whenever $|L| < 2^B$, where g bounds the global messages introduced below.

Some messages are not emitted by any witness: they belong to the system as a whole and are determined by the public statement. In ZisK these *global messages* open and close the continuation chains, initialise the registers and inject the public outputs into the buses. We model them as a fixed function $\mathcal{G}$ that maps every public statement $\boldsymbol{p} \in \mathcal{P}$ to an $\mathbb{F}$-weighted multiset of messages $\mathcal{G}(\boldsymbol{p})$, with $|\mathcal{G}(\boldsymbol{p})| \le g$ for a constant g.

Definition 2.10 (Balance of a Witness Collection). The *signed multiplicity* of a typed witness collection L is the $\mathbb{F}$-weighted multiset

$$\mathcal{M}_L := \sum_{i=1}^{|L|} \mathrm{Msg}_{\tau_i}(w_i),$$

that is,

$$\mathcal{M}_L(b, v) = \sum_{i=1}^{|L|} \sum_{\iota \in \mathcal{I}_{\tau_i}, b_\iota = b} \sum_{r=1}^{N_{\tau_i}} m_\iota(w_i, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w_i, r) = v].$$

The collection is *balanced for* $\boldsymbol{p} \in \mathcal{P}$, written $\mathrm{Balanced}_{\boldsymbol{p}}(L)$, if $\mathcal{M}_L + \mathcal{G}(\boldsymbol{p}) = 0$, i.e. $\mathcal{M}_L(b, v) + \mathcal{G}(\boldsymbol{p})(b, v) = 0$ for every bus $b \in \mathcal{B}$ and every message $v \in \mathbb{F}^{\ell_b}$. Note that $|\mathcal{M}_L + \mathcal{G}(\boldsymbol{p})| \le \Lambda(L) + g$.

Note that multiplicities are field elements: $\mathrm{Balanced}_{\boldsymbol{p}}(L)$ does not by itself assert that every message is sent and received the same number of times as integers, since p copies of a message cancel. We return to this point in Part II.

2.3. **Interfaces.**

A *non-interactive argument system* $\Pi = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ consists of a *prover* $\mathbf{P}$, which on input a statement $\mathbb{x}$ and a witness $\mathbb{w}$ outputs a proof $\pi \leftarrow \mathbf{P}(\mathbb{x}, \mathbb{w})$, and a deterministic *verifier* $\mathbf{V}$, together with a verification key vk_Π produced at setup, where $\mathbf{V}(\mathsf{vk}_\Pi, \mathbb{x}, \pi) \in \{0, 1\}$ and the value 1 means that the proof π is accepted for the statement $\mathbb{x}$. We write $\mathbf{V}_X(\mathbb{x}, \pi) := \mathbf{V}(\mathsf{vk}_X, \mathbb{x}, \pi)$ for the verifier of the system attesting to the relation $\mathcal{R}_X$, and keep the explicit form $\mathbf{V}(\mathsf{vk}, \mathbb{x}, \pi)$ where the key is a variable. The relations of the next section (Section 3.3) refer to argument systems only through their verifiers, so the prover is never named again, and to the following fixed public maps only through their signatures. Their security properties are defined in Section 2.4, and ZisK's choices are given in Section 3.4. Two of the maps are commitments, in the following sense.

Definition 2.11 (Deterministic Commitment Scheme). A *deterministic commitment scheme* Com consists of a set of *messages*, a finite set of *commitments*, and a polynomial-time computable function Commit from the former to the latter. It is *binding* if for every PPT adversary $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}(\mathcal{A}) := \Pr\big[(m, m') \leftarrow \mathcal{A}\big(1^\lambda\big) : m \neq m' \wedge \mathsf{Commit}(m) = \mathsf{Commit}(m')\big] \leq \mathsf{negl}(\lambda).$$

Since Commit is deterministic, binding is collision resistance of Commit. When the messages are vectors or matrices over $\mathbb{F}$, the scheme is a *vector commitment* if it also offers openings of individual positions.

The following interface elements are used throughout:

- **Carriers.** A finite key space $\mathcal{X}$ and an abelian group $G_{\mathcal{F}}$ of fingerprint values (Definition 2.8), and a finite abelian *accumulator group* $G_{\mathcal{H}}$.
- **Trace commitment.** A deterministic commitment scheme Com with message space $\bigcup_{\tau \in [M]} \mathcal{W}_\tau$ and commitment space $\{0,1\}^*$. A commitment $\mathrm{com} = \mathsf{Commit}(w)$ covers the trace and the AIR values of a witness w. The argument system for a chip takes the key $\chi \in \mathcal{X}$ as a public input, so the witness is key-independent.
- **Cached-column commitment.** A deterministic commitment scheme $\mathsf{Com}_{\mathrm{cach}}$ with message space $\bigcup_{\tau \in [M]} \mathbb{K}^{N_\tau \times |J^{\mathrm{cach}}_\tau|}$ and commitment space $\mathbb{F}^m$. Its output for the cached columns of a type is not part of a verification key but of the public statement (Definition 2.2).
- **Hash-to-group and multiset hash.** $h : [M] \times \{0,1\}^* \to G_{\mathcal{H}}$ maps a *typed commitment* (τ, com) to a group element, and

$$\mathcal{H}(\mathcal{S}) := \sum_{x \in \ \mathrm{supp}\ \mathcal{S}} \mathrm{mult}_{\mathcal{S}}(x) \cdot h(x) \in G_{\mathcal{H}}$$

  is the induced multiset hash on finite multisets $\mathcal{S}$ of typed commitments, where $\mathrm{mult}_{\mathcal{S}}(x)$ is the multiplicity of x in $\mathcal{S}$. $\mathcal{H}$ is additively homomorphic over multiset union.
- **Key derivation.** $\mathsf{DeriveKey} : \mathcal{P} \times G_{\mathcal{H}} \to \mathcal{X}$, a fixed public function of the public statement and the accumulated value.

2.4. **Security Definitions.**

Definition 2.12 (Knowledge Soundness). A non-interactive argument system Π for a relation $\mathcal{R}$ is *knowledge-sound* if there exists a PPT algorithm $\mathcal{E}$, the *extractor*, such that for every PPT adversary $\mathcal{A}$:

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A}) := \Pr\big[(\mathbb{x}, \pi) \leftarrow \mathcal{A}\big(1^\lambda\big); \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi) : \mathbf{V}(\mathsf{vk}_\Pi, \mathbb{x}, \pi) = 1 \wedge (\mathbb{x}; \mathbb{w}) \notin \mathcal{R}\big] \leq \mathsf{negl}(\lambda),$$

the probability taken over the coins of $\mathcal{A}$ and $\mathcal{E}$ and the random oracles, to which all algorithms have access. We assume that the running time of $\mathcal{E}(\mathbb{x}, \pi)$ is at most $\mathsf{Time}\big(\mathcal{A}(1^\lambda)\big) + \mathsf{poly}(\lambda)$ for every $\mathcal{A}$ and every $(\mathbb{x}, \pi)$ in the support of $\mathcal{A}$'s output: the extractor incurs an additive polynomial overhead over one evaluation of $\mathcal{A}$. We call $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A})$ the *knowledge-soundness error* of $\mathcal{A}$ against Π.

Remark (Straight-Line Extraction). *The extractor of Definition 2.12 is straight-line: it receives only the statement and the proof and does not rewind the adversary. For succinct proofs this is not achievable in the plain model, and in the ROM it holds for one argument but does not compose across a recursion (Remark 2.1). The definition is a proxy that makes the reduction structure explicit; a concrete instantiation replaces it by ROM extractability with the overhead discussed there.*

Definition 2.13 (Straight-Line Extractable Commitment). A deterministic commitment scheme Com (Definition 2.11) is *straight-line extractable* if there is a PPT algorithm $\mathcal{E}_{\mathsf{Com}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value com, outputs a message or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value com and at the end outputs a message m,

$$\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{A}) := \Pr[\mathsf{Commit}(m) = \mathrm{com} \wedge \mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{\le t}, \mathrm{com}) \neq m] \le \mathsf{negl}(\lambda),$$

where t is the time of the announcement and $\mathcal{T}_{\le t}$ the transcript up to it.

For a Merkle tree over a random oracle the extractor recomputes the leaves from the tree of queried nodes below the root; it fails only if the adversary later opens the root to a leaf it had not queried, which requires guessing an oracle output or finding a collision. Straight-line extractability implies binding: if $\mathcal{A}$ outputs $m \neq m'$ with $\mathsf{Commit}(m) = \mathsf{Commit}(m') = \mathrm{com}$, the extractor's output on com differs from at least one of them, so the adversary $\mathcal{D}$ that runs $\mathcal{A}$, announces com and returns one of m, m' chosen at random fails extraction with at least half of $\mathcal{A}$'s success probability: $\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}(\mathcal{A}) \le 2 \cdot \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D})$.

Definition 2.14 (Preimage Awareness of the Accumulator). The multiset hash $\mathcal{H}$ is *preimage aware on multisets of cardinality below* 2^B [DRS09] if there is a PPT algorithm $\mathcal{E}_{\mathcal{H}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value $y \in G_{\mathcal{H}}$, outputs a multiset of typed commitments or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value y and at the end outputs a multiset $\mathcal{S}$,

$$\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{A}) := \Pr[|\mathcal{S}| < 2^B \wedge \mathcal{H}(\mathcal{S}) = y \wedge \mathcal{E}_{\mathcal{H}}(\mathcal{T}_{\le t}, y) \neq \mathcal{S}] \le \mathsf{negl}(\lambda),$$

where t is the time of the announcement and $\mathcal{T}_{\le t}$ the transcript up to it.

The definition mirrors Definition 2.13, with the accumulator in the role of the commitment: what is announced is a value of $G_{\mathcal{H}}$, and what is extracted is the small multiset the adversary later opens it to. The cardinality restriction is necessary for ZisK's accumulator group: $\mathbb{Z}_p^n$ has exponent p, so adding p copies of any element to a multiset leaves $\mathcal{H}$ unchanged, and $2^B < p$ rules this out. Preimage awareness implies collision resistance on such multisets, since two distinct small multisets with the same hash would both have to equal the extractor's output.

For LtHash, collision resistance on bounded multisets reduces to the short-integer-solution problem [Ajt96, Zis25]; the reduction requires the inner digest of h to be collision resistant on its own, which a single field element is not, so we assume the digest has at least 2λ bits (Part IV). Preimage awareness, however, is not known to follow from any standard assumption for an algebraic accumulator, and assuming it is the second idealization of Remark 2.1. It is nevertheless the property the analysis needs, and collision resistance alone would not do: an adversary may fix a target y by querying $\mathsf{DeriveKey}$ at it and only afterwards obtain the h-values of the elements

it assembles into a multiset hashing to y, choosing which fresh values to combine after seeing them. Whether this succeeds is a preimage question, computational rather than statistical, and it is exactly what the extractor of Definition 2.14 rules out.

Definition 2.15 (Random-Oracle Model). The hash-to-group function h, the challenge derivation DeriveKey and the hash function underlying Com are independent random oracles. An adversary makes at most Q queries in total; every algorithm, including verifiers and extractors, has oracle access.

The definitions above, together with the random-oracle model, are the setting of the whole analysis. Two of the properties it relies on are idealizations, which we make explicit.

Remark 2.1 (Idealized Proof Model). *The analysis is idealized in three respects.*

First, Definition 2.12 postulates a straight-line extractor that runs in essentially the time of the adversary. Because the argument systems are hash-based, the natural model is the random oracle model (ROM), in which such extractors exist for a single argument [BCS16]. Composing straight-line extraction across the layers of a recursion tree, however, requires every argument to remain knowledge-sound relative to the same oracle, and such relativized succinct arguments do not exist in general [BCG24]. In our analysis we idealize over this obstruction and assume that straight-line extraction composes.

Second, the bus argument is sound only if the multiset of typed commitments that the accumulator $\boldsymbol{c}$ stands for is fixed before the random oracle answers at $\boldsymbol{c}$. For the trace commitments this is the straight-line extractability of Merkle trees in the ROM (Definition 2.13). For the accumulator we assume the analogous property, preimage awareness (Definition 2.14): a straight-line extractor recovers, from the transcript at the time a value is announced, the multiset the adversary later opens it to. For the algebraic accumulator $\mathcal{H}$ this is an idealization. Distinct small multisets with equal hash exist in abundance, only the hardness of finding them protects the scheme, and recovering a multiset from its accumulator is a subset-sum problem, so no straight-line extractor is known; only collision resistance follows from standard assumptions. Part II discusses the rigorous alternative.

Third, while the analysis produces explicit advantage bounds and reduction running times (Part II), the two idealizations mean that these should be read as statements about the reduction structure (the recursion topology is sound and the relations compose) rather than as a concrete security level.

3. The ZisK Proof System

This section describes the system that the rest of the paper analyzes: the aggregation tree into which a ZisK proof is organized, the three accumulators that its nodes carry, the relation attested by each kind of node, and the concrete choices ZisK makes for the components of Section 2. Part I proves knowledge soundness for the argument systems that attest to these relations, Part II shows what an accepting proof of the whole tree implies, and Part III gives the result its meaning in terms of program execution.

3.1. Structure.

A ZisK proof is a tree. Its leaves are the AIR instances of an execution: for each type $\tau \in [M]$ the prover produces some number of witnesses in $\mathcal{W}_\tau$, one per segment of that type, and proves each with an argument system for the AIR of the type. Directly above every AIR proof sits a *leaf verifier*, which verifies the AIR proof and re-exposes its statement in a common format. Above the leaf verifiers, *aggregation nodes* have A child slots each, where A is the aggregation arity. A slot holds either a child proof or a *null child*, a placeholder that is not verified and contributes nothing, and at least two slots of every aggregation node hold proofs. The tree need not be balanced,

because a node is proved as soon as enough children are available. At the top, a single *Final Verifier* verifies the root of the aggregation tree and performs the global checks; its statement, the external statement $\boldsymbol{p}_{\text{ext}}$, is the only thing an external verifier sees. Figure 2 shows an example with three leaves. The leaf verifiers, the aggregation nodes and the Final Verifier are the *recursive verifiers*: each is itself an AIR, whose constraints encode a verifier, and is proved with the same argument system as the chips (Part I).

Three quantities travel up the tree and are combined at every aggregation node: the number of leaves, an accumulation of the leaves' commitments, and an accumulation of the leaves' bus contributions. They are the *accumulators* of Section 3.2; the counter is range-checked at every node, and the Final Verifier checks one condition on each.

The order of operations matters for the bus argument. Bus contributions are fingerprints under a key χ, and the key must be fixed before any leaf is proved yet must not be under the prover's control. ZisK derives it from the leaves themselves: the prover commits to every witness, accumulates the typed commitments with the multiset hash $\mathcal{H}$, derives χ from the public statement and the accumulated value with $\mathsf{DeriveKey}$, and only then proves the leaves with χ as a public input. The tree recomputes the accumulation as it aggregates, and the Final Verifier checks that the key the leaves used is the one derived from what was actually aggregated. Because the shape of the tree is not known when the key is derived, the accumulation must not depend on it; this is why the accumulators below are commutative monoids.

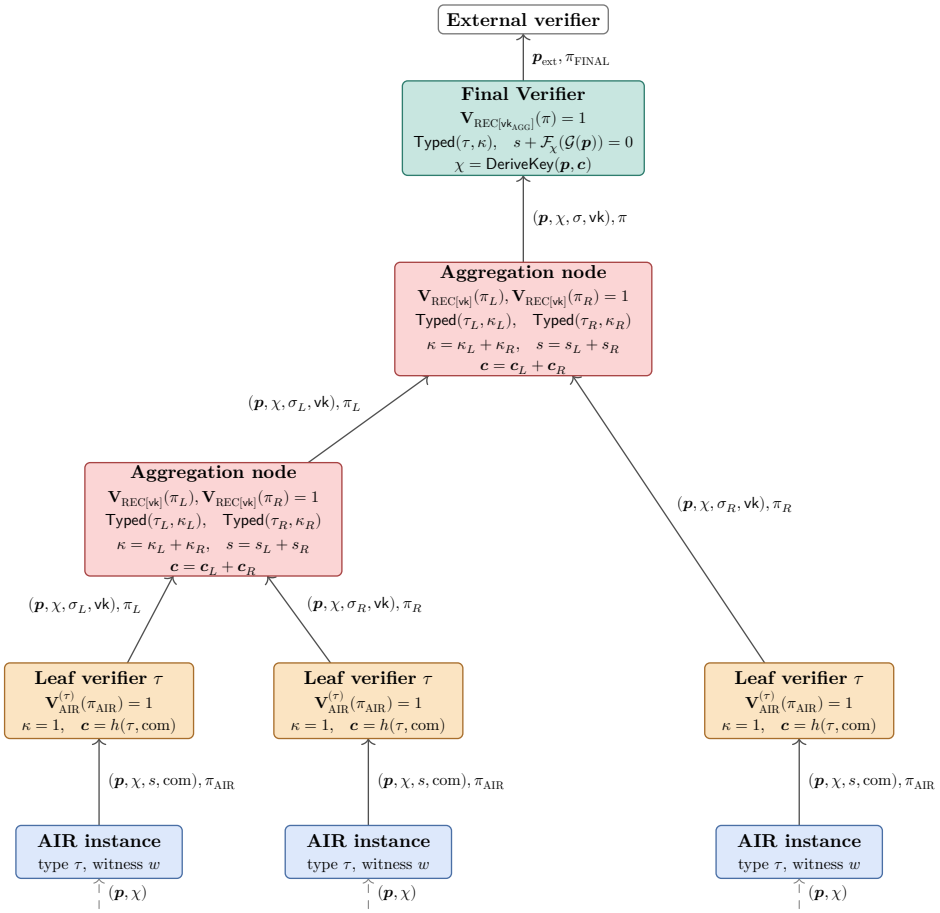


FIGURE 2. An aggregation tree with three AIR instances. Each arrow carries the statement and the proof a node produces, which the node above takes as witness: an AIR instance of type τ proves $(\boldsymbol{p}, \chi, s, \text{com})$; a leaf verifier re-exposes it as $(\boldsymbol{p}, \chi, \sigma, \mathsf{vk})$ with $\sigma = (\tau, 1, \boldsymbol{c}, s)$; an aggregation node adds its children's accumulators, dispatches on their types and declares type $M + 1$; the Final Verifier fixes $\mathsf{vk} = \mathsf{vk}_{\text{AGG}}$, checks the root predicates and exposes only $\boldsymbol{p}_{\text{ext}}$. The dashed arrows are the public inputs common to every leaf. Aggregation nodes are drawn with two children; in general they have A slots, the unused ones holding null children.

3.2. **Accumulators.**

Definition 3.1 (Accumulator). An *accumulator* over the aggregation tree consists of a commutative monoid of *values*, a *leaf contribution* assigning a value to every leaf, a *child check*, a predicate on values that every aggregation node evaluates on the value of each of its children, and a *root check*, a predicate on values that the Final Verifier evaluates on the value of the root of the aggregation tree. The value of a leaf is its contribution, and the value of an aggregation node is the sum of the values of its non-null children. By associativity and commutativity, the value of any node is the sum of the contributions of the leaves below it, whatever the shape of the subtree.

The recursion carries three accumulators. Their values at a node, together with the node's type, form the *recursive statement*

$$\sigma = (\tau, \kappa, \boldsymbol{c}, s),$$

where the type $\tau \in [M+1]$ is the type of the verified chip for a leaf verifier and the marker $M+1$ for an aggregation node. We write $\sigma.\tau$, $\sigma.\kappa$, $\sigma.\boldsymbol{c}$ and $\sigma.s$ for the components of a recursive statement.

Accumulator	**Monoid**	**Leaf contribution**	**Child check**	**Root check**
leaf counter κ	$(\mathbb{F}, +)$	1	$\kappa_X \in [1, 2^B)$	$\kappa \in [1, 2^B)$
challenge accumulator $\boldsymbol{c}$	$(G_{\mathcal{H}}, +)$	$h(\tau, \mathrm{com})$	—	$\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$
bus partial sum s	$(G_{\mathcal{F}}, +)$	$\mathcal{F}_\chi(\mathrm{Msg}_\tau(w))$	—	$s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$

TABLE 1. The three accumulators of the recursion.

The counter is added in $\mathbb{F}$. Its child check is the following predicate on a node's type and counter, which every aggregation node applies to each non-null child and the Final Verifier applies to the root.

Definition 3.2 (Typing Predicate). For a type τ and a counter κ,

$$\mathsf{Typed}(\tau, \kappa) := (\tau \in [M] \wedge \kappa = 1) \vee (\tau = M+1 \wedge 2 \le \kappa < 2^B).$$

The typing predicate places every counter in $[1, 2^B)$, and $A \cdot 2^B \le p$ then rules out wrap-around, so the field sums are integer sums and no node has 2^B or more leaves below it. The other two accumulators are checked only at the root. The value of the challenge accumulator at a node is $\mathcal{H}$ of the multiset of typed commitments below it, by the homomorphism of $\mathcal{H}$; the value of the bus partial sum is $\mathcal{F}_\chi$ of the summed message multisets below it, by the additivity of $\mathcal{F}$, so at the root it equals $\mathcal{F}_\chi(\mathcal{M}_L)$ for the collection L of all leaves. The Final Verifier adds the contribution $\mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p}))$ of the global messages, which it computes itself from the public statement and the key, so its root check $s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$ is a fingerprint test on $\mathcal{M}_L + \mathcal{G}(\boldsymbol{p})$.

An update rule that is not associative and commutative would make the root value depend on the shape of the tree, which is not known when the key is derived (Section 3.1); a hash-tree accumulation $\boldsymbol{c} = H(\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3)$, for instance, would force the prover to fix the tree before deriving χ. Writing every update as the monoid sum therefore loses no generality.

3.3. Recursive Relations.

We write a relation as $\mathcal{R} = \{(\mathbb{x}; \mathbb{w}) \mid \varphi(\mathbb{x}, \mathbb{w})\}$ with public statement $\mathbb{x}$, witness $\mathbb{w}$ and *defining predicate* φ; in the definitions below, φ is the conjunction of the clauses displayed to the right of the bar. Every node statement below the Final Verifier carries the public statement $\boldsymbol{p} \in \mathcal{P}$, a key $\chi \in \mathcal{X}$, a recursive statement σ and a verification key vk. We proceed from the leaves to the root, naming after each relation the argument system that proves it, since the relation above uses its verifier.

Definition 3.3 (AIR Relation). For $\tau \in [M]$,

$$\mathcal{R}^{(\tau)}_{\mathrm{AIR}} = \left\{ (\boldsymbol{p}, \chi, s, \mathrm{com}); w \;\middle|\; \begin{array}{ll} & \mathrm{Local}_\tau(\boldsymbol{p}, w) \\ \wedge & s = \mathrm{BusCont}_\chi(\mathcal{I}_\tau, w) \\ \wedge & \mathsf{Commit}(w) = \mathrm{com} \end{array} \right\}$$

with $\boldsymbol{p} \in \mathcal{P}$, $\chi \in \mathcal{X}$, $s \in G_{\mathcal{F}}$ and $w = (\boldsymbol{T}, \boldsymbol{v}) \in \mathcal{W}_\tau$.

The relation is in *commit-and-prove* form: the trace commitment is part of the statement. A proof for $\mathcal{C}_\tau$ already contains this commitment, so $\mathbf{V}^{(\tau)}_{\mathrm{AIR}}$ simply reads it off the proof and compares it with the statement component; the point of exposing it is that the leaf verifier can then bind the

challenge contribution to a value that is a function of the statement and witness rather than of an object that occurs in neither.

Let $\Pi^{(\tau)}_{\mathrm{AIR}}$ be an argument system for $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$, one per type, with verifier $\mathbf{V}^{(\tau)}_{\mathrm{AIR}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$.

Definition 3.4 (Leaf-Verifier Relation). For $\tau \in [M]$,

$$\mathcal{R}^{(\tau)}_{\mathrm{LEAF}} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); (\mathrm{com}, \pi_{\mathrm{AIR}}) \;\middle|\; \begin{array}{ll} & \sigma = (\tau, 1, \boldsymbol{c}, s) \\ \wedge & \mathbf{V}^{(\tau)}_{\mathrm{AIR}}((\boldsymbol{p}, \chi, s, \mathrm{com}), \pi_{\mathrm{AIR}}) = 1 \\ \wedge & \boldsymbol{c} = h(\tau, \mathrm{com}) \end{array} \right\}$$

The key vk carried in the statement is the propagated aggregation key, which is not used here.

In the terms of Section 3.2, a leaf verifier of type τ contributes 1, $h(\tau, \mathrm{com})$ and the bus contribution s certified by the AIR proof. Let $\Pi^{(\tau)}_{\mathrm{LEAF}}$ be an argument system for $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$, with verifier $\mathbf{V}^{(\tau)}_{\mathrm{LEAF}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{LEAF}}$; it hard-codes $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$.

An aggregation node must verify children of every kind, leaf verifiers of any type and other aggregation nodes. It hard-codes the M leaf-verifier keys and selects among them by the child's type; for an aggregation child it uses a key that it does not hard-code but reads from its own statement. The reserved type 0 marks a null child.

Definition 3.5 (Recursive Dispatch). For a type $\tau \in [M+1]$ and a key vk, the *dispatched verifier* is

$$\mathbf{V}^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}](\mathbb{x}, \pi) := \begin{cases} \mathbf{V}^{(\tau)}_{\mathrm{LEAF}}(\mathbb{x}, \pi) \text{ if } \tau \in [M], \\ \mathbf{V}(\mathsf{vk}, \mathbb{x}, \pi) \quad \text{if } \tau = M+1, \end{cases}$$

and $\mathbf{V}^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}]$ rejects everything if $\tau \notin [M+1]$; in particular a null child ($\tau = 0$) never passes the dispatched verifier.

The dispatch is a function of the key vk carried in the statement; the typing predicate of Definition 3.2 is what makes the counter meaningful.

Definition 3.6 (Aggregation Relation). Write $\mathbb{x}_X := (\boldsymbol{p}, \chi, \sigma_X, \mathsf{vk})$ for the statement of the child in slot $X \in [A]$, $\sigma_X = (\tau_X, \kappa_X, \boldsymbol{c}_X, s_X)$ for its recursive statement, and $N := \{X \in [A] : \tau_X \neq 0\}$ for the set of non-null slots. Then

$$\mathcal{R}_{\mathrm{AGG}} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); ((\sigma_X, \pi_X))_{X \in [A]} \;\middle|\; \begin{array}{ll} & \sigma = (M+1, \kappa, \boldsymbol{c}, s) \text{ and } |N| \geq 2 \\ \wedge & \mathsf{Typed}(\tau_X, \kappa_X) \text{ for } X \in N \\ \wedge & \mathbf{V}^{(\tau_X)}_{\mathrm{REC}}[\mathsf{vk}](\mathbb{x}_X, \pi_X) = 1 \text{ for } X \in N \\ \wedge & \kappa = \sum_{X \in N} \kappa_X \text{ and } \boldsymbol{c} = \sum_{X \in N} \boldsymbol{c}_X \text{ and } s = \sum_{X \in N} s_X \end{array} \right\}$$

Null slots carry no constraint: their proofs are not verified and their recursive statements do not enter the sums.

Two invariants of this relation are used repeatedly. First, the *propagation invariant*: every non-null child is verified on a statement that carries the parent's own $(\boldsymbol{p}, \chi, \mathsf{vk})$, so every node of a valid tree carries the root's public statement, fingerprint key and verification key; in particular an aggregation child is verified under $\mathbf{V}(\mathsf{vk}, \cdot, \cdot)$ for the very vk the parent declares. Since the cached-column commitments travel in $\boldsymbol{p}$ (Definition 2.2), the binding of $\mathsf{Commit}_{\mathrm{cach}}$ then makes the cached columns of all leaves of one type equal. Second, the *counter invariant*: Typed places the counters of the non-null children in $[1, 2^B)$, there are at least two and at most A of them, and $A \cdot 2^B \leq p$ makes $\kappa = \sum_{X \in N} \kappa_X$ an identity of integers; hence every non-null child has $\kappa_X \leq \kappa - 1$.

Let Π_{AGG} be an argument system for $\mathcal{R}_{\mathrm{AGG}}$, with verifier $\mathbf{V}_{\mathrm{AGG}}$ and key $\mathsf{vk}_{\mathrm{AGG}}$. There is no circularity: $\mathcal{R}_{\mathrm{AGG}}$ is one ordinary relation over statements that contain an arbitrary string vk, the key $\mathsf{vk}_{\mathrm{AGG}}$ of the argument system for that relation is produced afterwards at setup, and the Final relation below fixes $\mathsf{vk} \coloneqq \mathsf{vk}_{\mathrm{AGG}}$, at which point $\mathbf{V}_{\mathrm{REC}}^{(M+1)}[\mathsf{vk}_{\mathrm{AGG}}] = \mathbf{V}_{\mathrm{AGG}}$.

Definition 3.7 (Final Relation). Fix the aggregation key $\mathsf{vk}_{\mathrm{AGG}}$ as a parameter. Then, with $\boldsymbol{p} \coloneqq (\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}})$,

$$\mathcal{R}_{\mathrm{FINAL}} = \left\{ \boldsymbol{p}_{\mathrm{ext}}; (\boldsymbol{p}_{\mathrm{pv}}, \chi, \sigma, \pi) \;\middle|\; \begin{array}{ll} & \sigma = (\tau, \kappa, \boldsymbol{c}, s) \text{ and } \mathsf{Typed}(\tau, \kappa) \\ \wedge & \mathbf{V}_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = 1 \\ \wedge & s + \mathcal{F}_{\chi}(\mathcal{G}(\boldsymbol{p})) = 0 \\ \wedge & \chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c}) \end{array} \right\}$$

Only the external statement $\boldsymbol{p}_{\mathrm{ext}}$ survives at the root: the proof values, the fingerprint key, the recursive statement and the child proof are witnesses. The typing predicate excludes a null child at the root, which would otherwise leave $\boldsymbol{c}$ and s unconstrained. Dispatching on τ lets the Final Verifier accept a leaf-verifier proof directly when the execution fits in a single AIR instance.

Let Π_{FINAL} be an argument system for $\mathcal{R}_{\mathrm{FINAL}}$, with verifier $\mathbf{V}_{\mathrm{FINAL}}$ and key $\mathsf{vk}_{\mathrm{FINAL}}$; this is the verifier run by the external party. The verification keys involved play different roles. Table 2 records them.

Key	**Selected by**	**Hard-coded in**
$\mathsf{vk}_{\mathrm{AIR}}^{(\tau)}$	the type τ of the leaf verifier	$\Pi_{\mathrm{LEAF}}^{(\tau)}$
$\mathsf{vk}_{\mathrm{LEAF}}^{(\tau)}$	the child type $\tau_X \in [M]$ read from the child statement	Π_{AGG} (all M keys)
$\mathsf{vk}_{\mathrm{AGG}}$	the child type $\tau_X = M + 1$; the key itself arrives as the public vk	Π_{FINAL} only
$\mathsf{vk}_{\mathrm{FINAL}}$	—	the external verifier

TABLE 2. Verification-key ledger. The type τ selects which hard-coded key an aggregation node uses for a child; the propagated public vk is the aggregation verifier's own key, which no aggregation node can hard-code and which the Final Verifier fixes.

3.4. ZisK Instantiation.

The security analysis is stated for the following choices, which fix the carriers and the maps of Section 2.3. Four of them are built from one *underlying hash function*, and the analysis depends on that function only through the random-oracle model of Definition 2.15, never through its internals. ZisK currently instantiates it with Poseidon2 [GKS23], whose arithmetisation-friendly design makes the recursive verifiers cheap; a general-purpose hash such as BLAKE3 [O'C+20] or SHA-2 [Nat15] is an equally valid instantiation, and switching among them changes none of the statements below. The only requirements are those the analysis names where it uses them: collision resistance of the digest, with at least 2λ bits (Part IV), and the random-oracle idealization.

Bus fingerprint. The key space is $\mathcal{X} = \mathbb{K}^2$, a key being a *challenge pair* $\chi = (\alpha, \gamma)$, and the values lie in $G_{\mathcal{F}} = \mathbb{K}$. The fingerprint is LogUp [Hab22]: for $\chi = (\alpha, \gamma)$ the message (b, v) is *compressed* to $\hat{e}_{\gamma}(b, v) \coloneqq b + \sum_{j=1}^{\ell_b} \gamma^j v_j \in \mathbb{K}$ and

$$\mathcal{F}_{\chi}^{\mathrm{LogUp}}(D) := \sum_{(b,v)\in \operatorname{supp} D} \frac{D(b,v)}{\alpha + \hat{e}_{\gamma}(b,v)},$$

with $\mathcal{F}_{\chi}^{\mathrm{LogUp}}(D) := \bot$ if some denominator vanishes. It admits every nonzero multiset, and Part II shows that it is statistically sound with $\varepsilon_{\mathcal{F}}(\Lambda) = \big(\ell \cdot \binom{\Lambda}{2} + \max(\Lambda - 1, 0)\big)/|\mathbb{K}|$. The AIR argument does not evaluate this closed form. It constrains a running sum over the rows of the trace, adding $m_{\iota}(w,r)/\big(\alpha + \hat{e}_{\gamma}(b_{\iota}, \boldsymbol{e}_{\iota}(w,r))\big)$ at every row through an explicitly constrained inverse, and exposes the final value s. Grouping the row terms by message shows that the exposed value equals $\mathcal{F}_{\chi}^{\mathrm{LogUp}}(\mathrm{Msg}_{\tau}(w))$ whenever every row denominator is nonzero. At a pole the constraints do not determine s: a row of zero multiplicity whose denominator vanishes satisfies its fraction constraint for every value of the fraction column. The poles of a witness are at most $N_{\tau}|\mathcal{I}_{\tau}|$ values of α for each γ, fixed by the witness, and the key is derived after the commitment to the witness; the probability of hitting one is charged in the binding lemma of Part I.

Multiset hash. The accumulator group is $G_{\mathcal{H}} = \mathbb{Z}_p^n$ with $n = 368$, and h and $\mathcal{H}$ are LtHash [Cla+03, Zis25]: h absorbs the typed commitment into a sponge of the underlying hash function and expands it, by 22 further permutations, into a pseudorandom vector of $\mathbb{Z}_p^n$ (the 23 states of width 16). The type enters h as the root of the type's fixed columns, which identifies the type provided that no two types have the same fixed columns, and $\mathcal{H}$ adds these vectors with multiplicity. Its collision resistance on multisets of cardinality below 2^B rests on the hardness of the short integer solution problem [Ajt96]; the analysis assumes the stronger preimage awareness (Definition 2.14), which is the second idealization of Remark 2.1. The cardinality bound is what the leaf counter enforces.

Key derivation. DeriveKey hashes the public statement $\boldsymbol{p}$ —the external statement and the proof values— and the accumulated vector with the underlying hash function into a *global challenge* in $\mathbb{K}$, from which a fresh sponge squeezes the pair (α, γ). The analysis treats the pair as a single random-oracle output (Definition 2.15) and suppresses the $(Q+1)/|\mathbb{K}|$ term for a derived α colliding with a previously queried input.

Trace commitment. Commit is a Merkle tree over the underlying hash function, so it is straight-line extractable in the random-oracle model (Definition 2.13).

Cached-column commitment. $\mathsf{Commit}_{\mathrm{cach}}$ is likewise a Merkle tree over the underlying hash function. The program ROM is the cached column group of the ROM chip, and its root is one of the publics, so a proof is bound to a program through $\boldsymbol{p}$. The leaf commitment com covers the witness columns and the AIR values of the first stage, not the cached columns; this is sound because DeriveKey absorbs $\boldsymbol{p}$, and with it the cached-column commitment, so the cached columns are fixed before the key χ is derived, exactly as the committed witnesses are.

References

Zis26. ZisK Core Team: ZisK Whitepaper, https://github.com/0xPolygonHermez/zisk-whitepaper

DRS09. Dodis, Y., Ristenpart, T., Shrimpton, T.: Salvaging Merkle-Damgård for Practical Applications. In: Advances in Cryptology – EUROCRYPT 2009. pp. 371–388. Springer, Berlin, Heidelberg (2009)

Ajt96. Ajtai, M.: Generating Hard Instances of Lattice Problems (Extended Abstract). In: 28th Annual ACM Symposium on Theory of Computing. pp. 99–108. ACM Press (1996)

Zis25. ZisK: Secure Challenge Derivation in ZisK, https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg

BCS16. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive Oracle Proofs. In: TCC 2016-B: 14th Theory of Cryptography Conference, Part II. pp. 31–60. Springer, Berlin, Heidelberg (2016)

BCG24. Barbara, A., Chiesa, A., Guan, Z.: Relativized Succinct Arguments in the ROM Do Not Exist, https://eprint.iacr.org/2024/728

GKS23. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A Faster Version of the Poseidon Hash Function. In: El Mrabet, N., De Feo, L., and Duquesne, S. (eds.) AFRICACRYPT 23: 14th International Conference on Cryptology in Africa. pp. 177–203. Springer, Cham (2023)

O'C+20. O'Connor, J., Aumasson, J.-P., Neves, S., Wilcox-O'Hearn, Z.: BLAKE3: One Function, Fast Everywhere, (2020)

Nat15. National Institute of Standards and Technology: Secure Hash Standard (SHS). (2015)

Hab22. Haböck, U.: Multivariate lookups based on logarithmic derivatives, https://eprint.iacr.org/2022/1530

Cla+03. Clarke, D.E., Devadas, S., Dijk, M. van, Gassend, B., Suh, G.E.: Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In: Laih, C.-S. (ed.) Advances in Cryptology – ASIACRYPT~2003. pp. 188–207. Springer, Berlin, Heidelberg (2003)